

Reviewer Pack — Minimal Rank of Quaternionic Forms and DPLL Proof Tree Height...

Entry 3a248e674f74 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 21:50:35 UTC

Minimal Rank of Quaternionic Forms and DPLL Proof Tree Height Inequality

Entry ID: 3a248e674f74 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 21:50:35 UTC

1. Conjecture

Field A (mathematical branch): Algebra (Quaternionic Forms) **Field B** (complexity object): Complexity Theory (DPLL Proof Complexity)

Statement:

The minimal rank of a quaternionic form representing the clause indicator polynomial for a given Boolean formula φ is bounded above by the height of the DPLL proof tree for φ , i.e., $\min \text{Rank}(Q\varphi) \leq \text{Height}(\text{DPLL}(\varphi))$ for all Boolean formulas φ .

Rationale (proposer's reasoning):

Quaternionic forms provide a non-commutative algebraic structure that could encode complexity-theoretic properties in their ranks. The conjecture posits that the rank of such forms is related to the structural complexity captured by DPLL proof trees, potentially revealing new insights into computational hardness.

Taxonomy category: QuaternionicForms (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 4de20a0964c51563

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a given Boolean formula φ with n variables, if the minimal rank of the quaternionic form $Q\varphi$ is less than or equal to the height of the DPLL proof tree for φ across at least 25 out of 30 random seeds, then support for the conjecture is provided.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal rank" quaternionic forms AND "DPLL proof tree height inequality" - "clause indicator polynomial" quaternionic forms AND DPLL complexity - algebraic representation quaternionic forms AND DPLL proof complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        # Find pivot row
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]

        # Eliminate non-pivot elements
        for j in range(n):
            if i != j:
                factor = Fraction(A[j][i], A[i][i])
                for k in range(n+1):
                    A[j][k] -= factor * A[i][k]

    # Back-substitute to find solution
    x = [0] * n
    for i in range(n-1, -1, -1):
        x[i] = Fraction(A[i][n], A[i][i])
        for j in range(i-1, -1, -1):
            A[j][n] -= A[j][i] * x[i]
    return x

def matrix_multiply(A, B):
    n = len(A)
    C = [[0] * n for _ in range(n)]
    for i in range(n):
```

```

        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def height_dp11(clauses):
    assignment = {}
    stack = []

    def backtrack():
        if all(l in assignment or -l in assignment for l in set.union(*clauses)):
            return True
        literal = next((l for l in set.union(*clauses) if l not in assignment and -l not in assignment))
        if literal is None:
            return False

        assignment[literal] = True
        stack.append(literal)

        if backtrack():
            return True

        del assignment[literal]
        stack.pop()

        assignment[-literal] = True
        stack.append(-literal)

        if backtrack():
            return True

        del assignment[-literal]
        stack.pop()

        return False

    return len(stack) if backtrack() else 0

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = random.randint(5, 40)
    clauses = []
    for _ in range(n):
        clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
        if len(set(clause)) > 1:
            clauses.append(tuple(sorted(clause)))

    Q = [[0] * n for _ in range(n)]
    for clause in clauses:
        for l in clause:
            Q[l-1][l-1] += Fraction(1, 2)
            for m in clause:
                if l != m:
                    Q[l-1][m-1] -= Fraction(1, 4)

```

```

rank_Q = len(gaussian_elimination(Q))
height_DPLL = height_dppll(clauses)

return {
    "metric_name": "Rank and DPLL Height",
    "metric_value": rank_Q,
    "instances_tested": n,
    "n_max": n,
    "conjecture_holds": rank_Q <= height_DPLL,
    "counterexample": "" if rank_Q <= height_DPLL else f"rank_Q={rank_Q}, height_DPLL={height_DPLL}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [random.randint(2, 1000003) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"rank_Q > height_DPLL\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_33ef30dc.py", line 121, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_33ef30dc.py", line 98, in run_trial
    Q[l-1][l-1] += Fraction(1, 2)
    ~~~~~^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that it was unable to complete its execution and provide a result. | next: Review the test code for potential bugs or issues that may cause it to crash and fix them. Then re-run the test with a different set of seeds to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13625
2	preregistration	ollama_remote	glm4:latest	0	0	9955
3	novelty	ollama_remote	glm4:latest	0	0	8390
4	novelty	ollama_remote	glm4:latest	0	0	8923
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17456
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	97759
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20674
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16531
9	judge	ollama_remote	glm4:latest	0	0	74595

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 267906 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/3a248e674f74.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/3a248e674f74.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/3a248e674f74.tar.gz (if generated)